

Data mining practice9 - 벼리

선형회귀

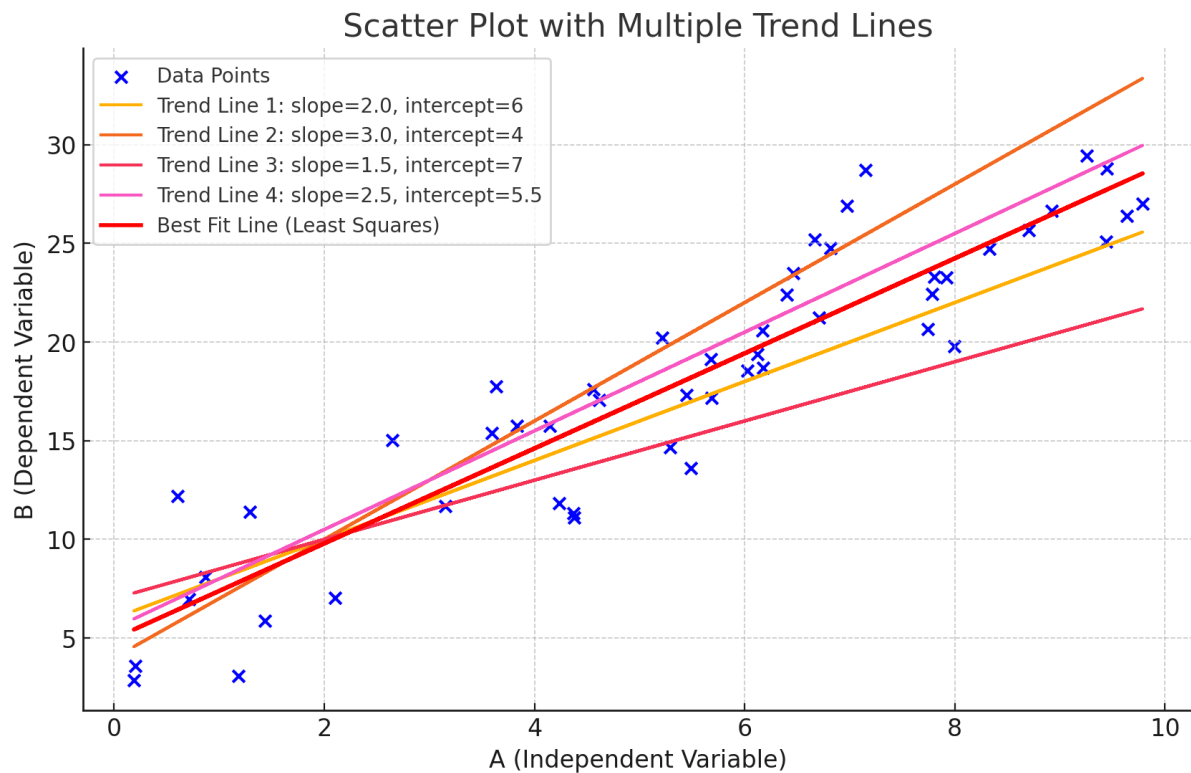
선형회귀 개념

선형 회귀는 독립 변수와 종속 변수 사이의 관계를 직선으로 나타내는 통계적 기법이다.
하나의 변수로 다른 변수를 예측하기 위한 방법

$$y = \beta_0 + \beta_1 x + \epsilon$$

여기서:

- y : 예측하려는 종속 변수
- x : 독립 변수
- β_0 : y 절편 (x 가 0일 때 y 의 값)
- β_1 : 기울기 (x 가 증가할 때 y 의 변화율)
- ϵ : 오차, 예측값과 실제 값의 차이



A와 B 변수에 대해, 각각의 데이터 값을 파란색 x로 표시하였다.

이 상황에서 우리는 선형 추세선을 다양하게 그릴 수 있는데, 이때 보다 데이터에 잘 맞고 예측 성공률을 높이기 위해서, **최소제곱법**을 사용한다.

최소제곱법은 **각 데이터 포인트와 회귀선 간의 오차를 최소화**하여 전체적으로 데이터에 가장 가까운 직선을 찾기 때문에 사용한다.

cost function

cost 함수는, 선형 회귀 식으로 예측한 값과 실제 값 사이의 오차의 제곱을 계산하여 모델의 성능을 평가하는 도구로 사용된다.

Gradient Descent

결론적으로는, 선형회귀식 $y = t_0 + t_1x$ 라는 식에서 t_0 과 t_1 을 찾아야 하고, 이를 위해 cost 함수, 최소제곱법 등을 사용한다.

Task 1: alpha를 변경해서 학습을 수행하고 67페이지와 같은 그래프를 그려라

```
exData <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/ex1data1.txt", header = F, col.names = c('pop', 'profit'))

theta0 = 0
theta1 = 1

h <- function(x, t0, t1){
  t0+t1*x
}

costJ <- function(t0, t1){
  m = nrow(exData)
  1/(2*m) * sum((h(exData$pop, t0, t1) - exData$profit) ** 2)
}

costJ(theta0, theta1)

costDF<-data.frame(t1 =seq(-1, 2, 0.1),
                   cost =seq(-1, 2, 0.1) %>%
                     sapply(function(x) { costJ(theta0, x)}))

costDF %>% ggplot(aes(x = t1, y = cost))+ geom_point()

gradient_theta0 <-function(t0, t1){
  mean(h(exData$pop, t0, t1) -exData$profit)}

gradient_theta1 <-function(t0, t1){
```

```

    mean((h(exData$pop, t0, t1) - exData$profit) * exData$pop))}

num_iter <- 1500
alpha <- 0.01

theta0 = 0
theta1 = 1

costDF = data.frame(iter = 0, t0 = theta0, t1 = theta1, cost = costJ(theta0,
theta1))

for( i in 1:num_iter) {
  theta0_update <- gradient_theta0(theta0, theta1)
  theta1_update <- gradient_theta1(theta0, theta1)

  theta0 <- theta0 - alpha * theta0_update
  theta1 <- theta1 - alpha * theta1_update

  costDF <- rbind(costDF, c(i, theta0, theta1, costJ(theta0,
theta1)))
}

costDF %>%
  ggplot(aes(x = iter, y = cost)) + geom_point(alpha = 0.5)

```

경사 하강법을 사용해 최적의 파라미터를 구하는 과정

```

# alpha 값 설정
alpha_values <- c(0.001, 0.005, 0.01, 0.05, 0.1)

```

alpha값을 초기에 다양하게 생성하여 벡터로 저장한 후, 각각에 대해 적용하며 학습을 확인한다

```

# 초기 theta 값 설정
theta0 <- 0
theta1 <- 1
num_iter <- 1500

# 데이터를 저장할 데이터프레임 초기화
cost_data <- data.frame()

# 여러 alpha 값에 대해 반복 실행
for (alpha in alpha_values) {
  theta0 <- 0
  theta1 <- 1
  temp_costDF <- data.frame(iter = 0, t0 = theta0, t1 = the
ta1, cost = costJ(theta0, theta1), alpha = alpha)

  for (i in 1:num_iter) {
    theta0_update <- gradient_theta0(theta0, theta1)
    theta1_update <- gradient_theta1(theta0, theta1)

    theta0 <- theta0 - alpha * theta0_update
    theta1 <- theta1 - alpha * theta1_update

    # 각 반복마다 Cost 값 저장
    temp_costDF <- rbind(temp_costDF, c(i, theta0, theta1,
costJ(theta0, theta1), alpha))
  }

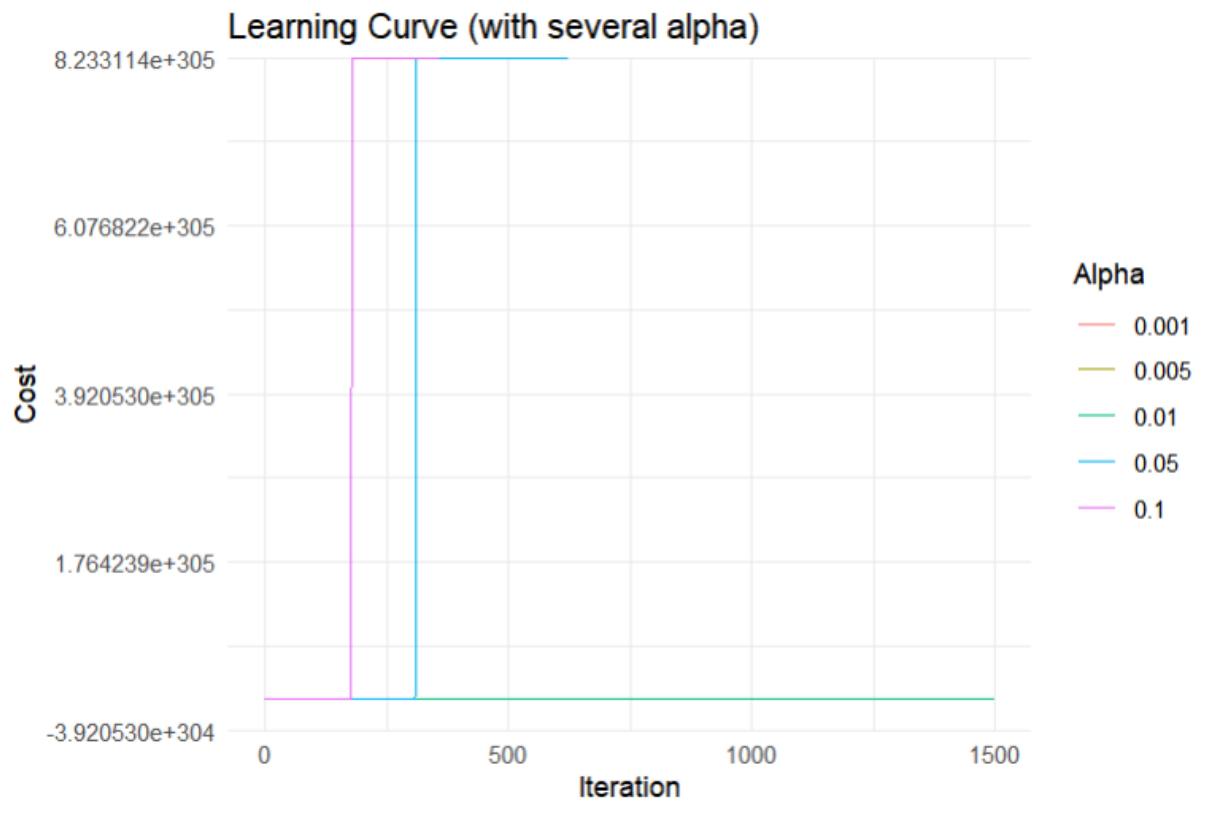
  cost_data <- rbind(cost_data, temp_costDF)
}

# 그래프 그리기
ggplot(cost_data, aes(x = iter, y = cost, color = as.factor
(alpha))) +
  geom_line() +
  labs(title = "Learning Curve (with several alpha)", x =

```

```
"Iteration", y = "Cost", color = "Alpha") +  
  theme_minimal()
```

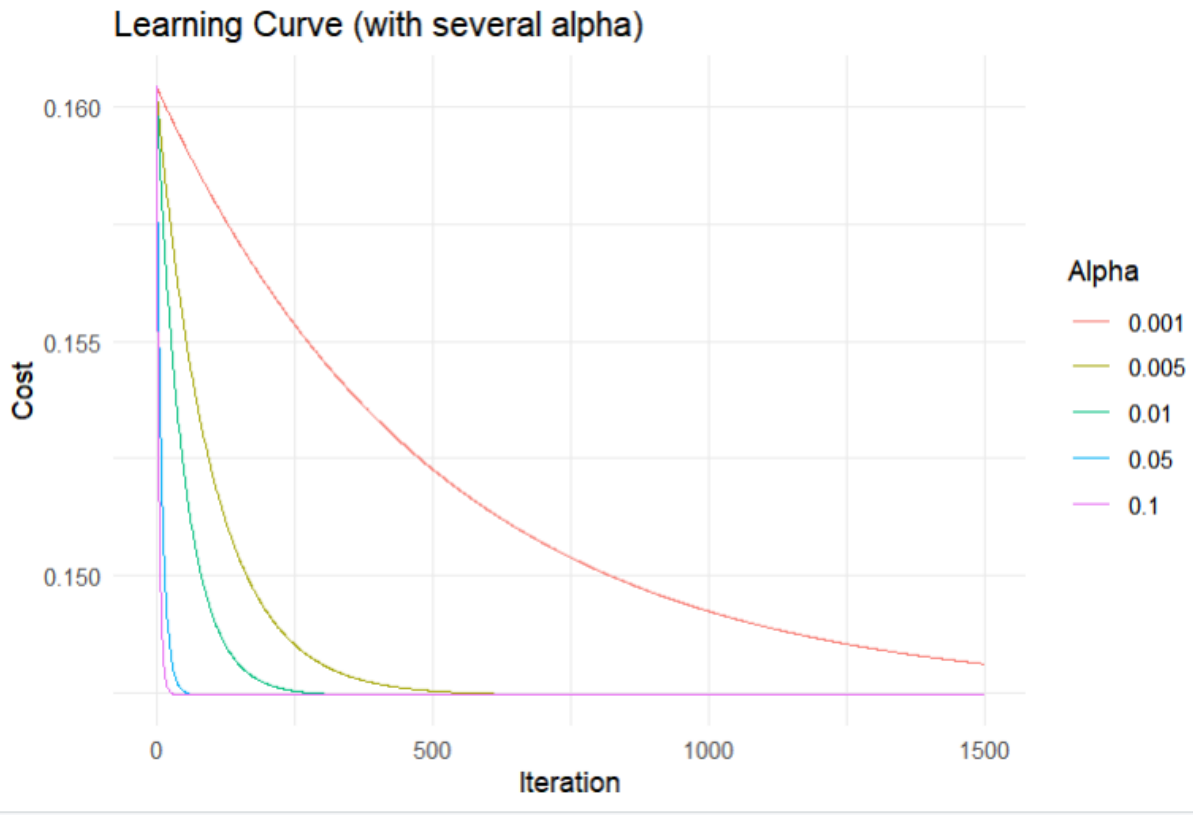
동일하게 cost값을 저장하고, 벡터로 지정한 각 alpha값에 따라 학습커브를 시각화, 학습 횟수에 따른 cost함수 값을 그래프로 표현함



그래프를 확인해 보면, 알파값이 낮은 0.001이나, 0.005는 확인되지 않고, iteration에 따라서 cost값이 점점 내려오는 지표를 확인할 수가 없다. 특히 cost값이 비정상적으로 커서 수렴하는 지점을 확인할 수가 없다. 이를 해결하기 위해

```
exData$pop <- scale(exData$pop)  
exData$profit <- scale(exData$profit)
```

실제 데이터의 scale을 조정하여 다시 시각화함



각 Alpha값 별로, 시행횟수에 따른 cost값이 다음과 같이 보여진다.

Task 2: Cost 감소가 멈추는 시점에서 학습을 중단하게끔 알고리즘을 수정하여라.
alpha값을 변경할 때, 학습에 소요된 iteration의 수(theta 업데이트 횟수)를 계산하여 표 혹은 그래프로 나타내어라.

cost감소가 멈추는 지점을 하나의 변수로서 정리하고, 그 값 아래로 떨어지면 수렴했다고 판단하여 학습을 중단하게 한다.

```
# alpha 값 설정
alpha_values <- c(0.001, 0.005, 0.01, 0.05, 0.1)
tolerance <- 1e-6 # Cost 변화가 이 값 이하로 작아지면 수렴했다고
```

판단

```
max_iter <- 1500 # 최대 반복 횟수 제한
```

```
# 각 alpha에 대해 경사 하강법 실행
for (alpha in alpha_values) {
  theta0 <- 0
  theta1 <- 1
  cost_prev <- costJ(theta0, theta1)

  for (i in 1:max_iter) {
    # 경사 하강법으로 theta0, theta1 업데이트
    theta0_update <- gradient_theta0(theta0, theta1)
    theta1_update <- gradient_theta1(theta0, theta1)

    theta0 <- theta0 - alpha * theta0_update
    theta1 <- theta1 - alpha * theta1_update

    # 새로운 Cost 계산
    cost_curr <- costJ(theta0, theta1)

    # 수렴 확인 (변화량이 tolerance보다 작아지면 중단)
    if (abs(cost_curr - cost_prev) < tolerance) {
      iteration_data <- rbind(iteration_data, data.frame(alpha = alpha, iterations = i))
      break
    }

    # 이전 Cost 값 갱신
    cost_prev <- cost_curr

    # 최대 반복 횟수까지 완료하면 수렴 실패로 간주
    if (i == max_iter) {
      iteration_data <- rbind(iteration_data, data.frame(alpha = alpha, iterations = max_iter))
    }
  }
}
```


동일하게 경사하강법을 통해 학습을 진행하고, cost감소가 tolerance보다 작아지면 수렴했다고 판단, 학습을 중단한다.

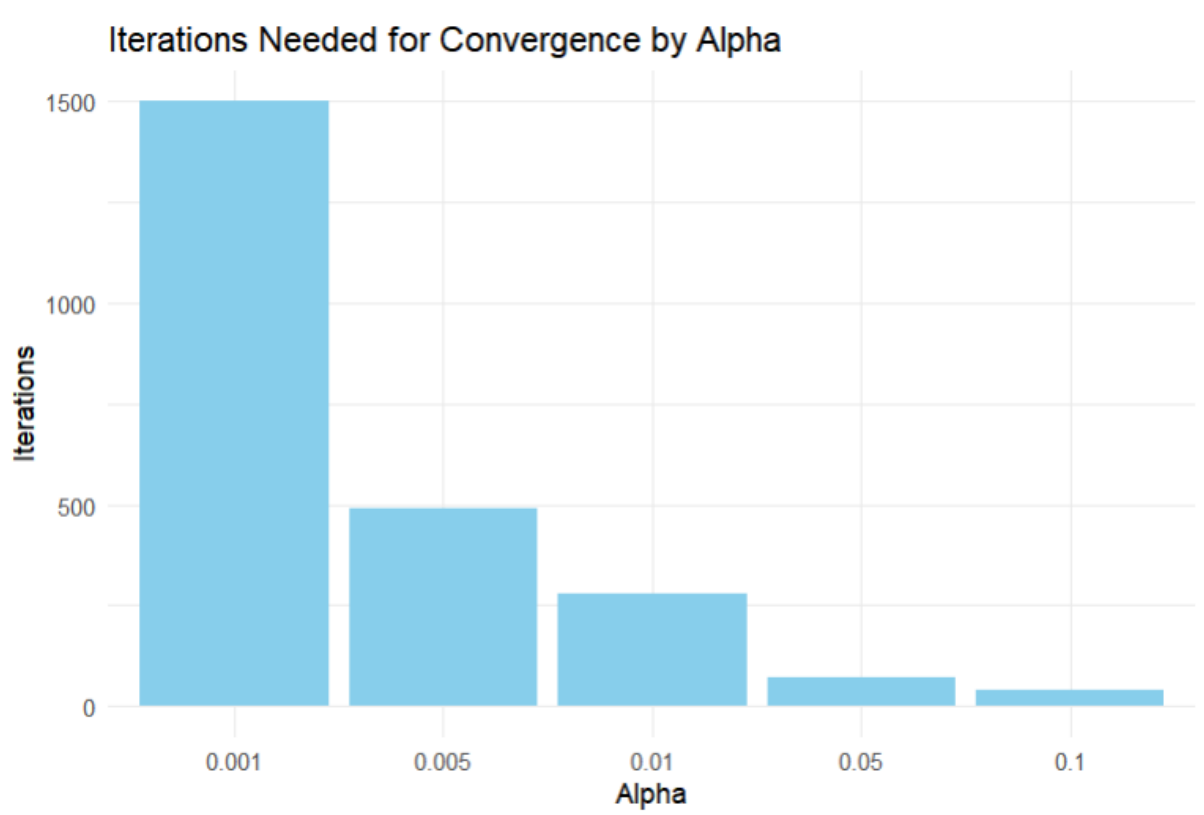
```
# iteration_data를 출력하여 확인
print(iteration_data)
```

학습의 소요된 iteration은 다음과 같다

```
> print(iteration_data)
  alpha iterations
1 0.001       1500
2 0.005        491
3 0.010        280
4 0.050         72
5 0.100         39
```

이를 그래프로 시각화하면 다음과 같다.

```
# iteration_data를 그래프로 시각화
ggplot(iteration_data, aes(x = as.factor(alpha), y = iterations)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  labs(title = "Iterations Needed for Convergence by Alpha", x = "Alpha", y = "Iterations") +
  theme_minimal()
```



Task 3: 강의자료 65페이지의 <fill in> 부분을 완성하고 정상적으로 학습이 되는 것을 확인하여라.

```
exData2 <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/
데이터 마이닝 실습/Data-Mining-Practicum/myR/ex1data2.txt", header = F, col.names = c('size', 'num_bedroom', 'price'))

mean1 <- mean(exData2$size)
sd1 <- sd(exData2$size)
mean2 <- mean(exData2$num_bedroom)
sd2 <- sd(exData2$num_bedroom)

exData2$size.norm <- (exData2$size - mean1) / sd1
exData2$num_bedroom.norm <- (exData2$num_bedroom - mean2) / sd2

featureDF <- exData2 %>%
  mutate(bias = 1)%>%
  select(bias, ends_with('norm'))
```

```

labelVector <- exData2$price
theta_vector <- c(0,0,0)

h <- function(x, theta_vector){
  theta_vector %*% x
}

costFun <- function(theta_vector){
  v <- as.matrix(featureDF) %*% theta_vector - labelVector
  (t(v) %*% v / (2*nrow(featureDF)))[1,1]
}
costFun(theta_vector)

costDF <- data.frame(iter = 0,
                     t0 = theta_vector[1],
                     t1 = theta_vector[2],
                     t2 = theta_vector[3],
                     cost = costFun(theta_vector))

num_iter = 1500
alpha <- 0.05
n <- nrow(fratureDF)

for( i in 1:num_iter){
  errors <- as.matrix(featureDF) %*% theta_vector - labelVector

  # 각 theta에 대한 기울기 계산
  theta_update <- alpha * (t(as.matrix(featureDF)) %*% errors) / n
  theta_vector <- theta_vector - theta_update
  costDF <- rbind(costDF, c(i, theta_vector, costFun(theta_vector)))
}

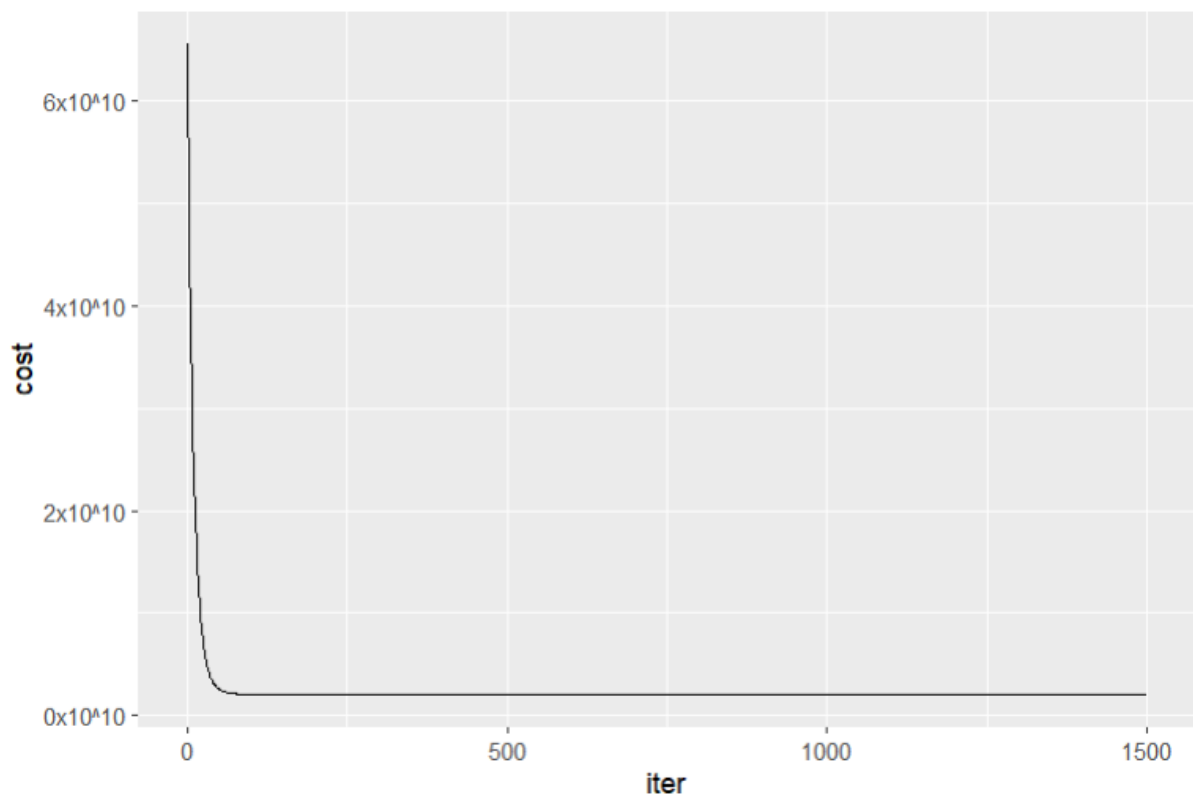
label_ko_num = function(num){

```

```

ko_num = function(x){
  new_num = x %/% 10**10
  return(paste(new_num, 'x10^10', sep = ''))
}
return(sapply(num, ko_num))
}
cosdDF %>% ggplot(aes(x = iter, y = cost))+
  geom_line()+
  scale_y_continuous(labels = label_ko_num)

```



Task 4: 직접 구현한 선형회귀 코드를 사용해서 부동산 가격을 예측하는 선형회귀 모델을 학습하여라

```

library(readxl)
estate <- read_excel("C:/Users/silkj/Desktop/한동대학교/5학기/
데이터 마이닝 실습/Data-Mining-Practicum/myR/Real estate valua
tion data set.xlsx")

```

```
head(estate)
colnames(estate) <- c("No", "transaction_date", "house_age", "distance_to_mrt",
                      "num_convenience", "latitude", "longitude", "price")
```

부동산 데이터를 입력하고, 각 변수 이름을 바꾸어 가시성을 높인다.

```
# 표준화(정규화) 적용
estate$house_age_norm <- (estate$house_age - mean(estate$house_age)) / sd(estate$house_age)
estate$distance_to_mrt_norm <- (estate$distance_to_mrt - mean(estate$distance_to_mrt)) / sd(estate$distance_to_mrt)
estate$num_convenience_norm <- (estate$num_convenience - mean(estate$num_convenience)) / sd(estate$num_convenience)
estate$latitude_norm <- (estate$latitude - mean(estate$latitude)) / sd(estate$latitude)
estate$longitude_norm <- (estate$longitude - mean(estate$longitude)) / sd(estate$longitude)
```

경사 하강법에 더욱 적합하도록 정규화를 적용. 데이터를 표준화하여 평균을 0, 표준편차를 1로 바꿈

```
# 특징 행렬 생성 (정규화된 변수와 bias 항 추가)
featureDF <- estate %>%
  mutate(bias = 1) %>%
  select(bias, house_age_norm, distance_to_mrt_norm, num_convenience_norm, latitude_norm, longitude_norm)

# 목표 변수 (라벨 벡터)
labelVector <- estate$price
# theta 초기화 (특징 개수에 맞춰 초기화)
theta_vector <- rep(0, ncol(featureDF))
```

특징 행렬 생성, 상수항을 추가하여 theta0의 절편 값을 계산할 수 있도록 함
select를 통해 상수항과, 정규변환된 열의 값들만 사용하도록

```
h <- function(x, theta_vector) {  
  theta_vector %*% x  
}
```

가설 함수 정의

```
# Cost 함수  
costFun <- function(theta_vector) {  
  v <- as.matrix(featureDF) %*% theta_vector - labelVector  
  (t(v) %*% v / (2 * nrow(featureDF)))[1, 1]  
}
```

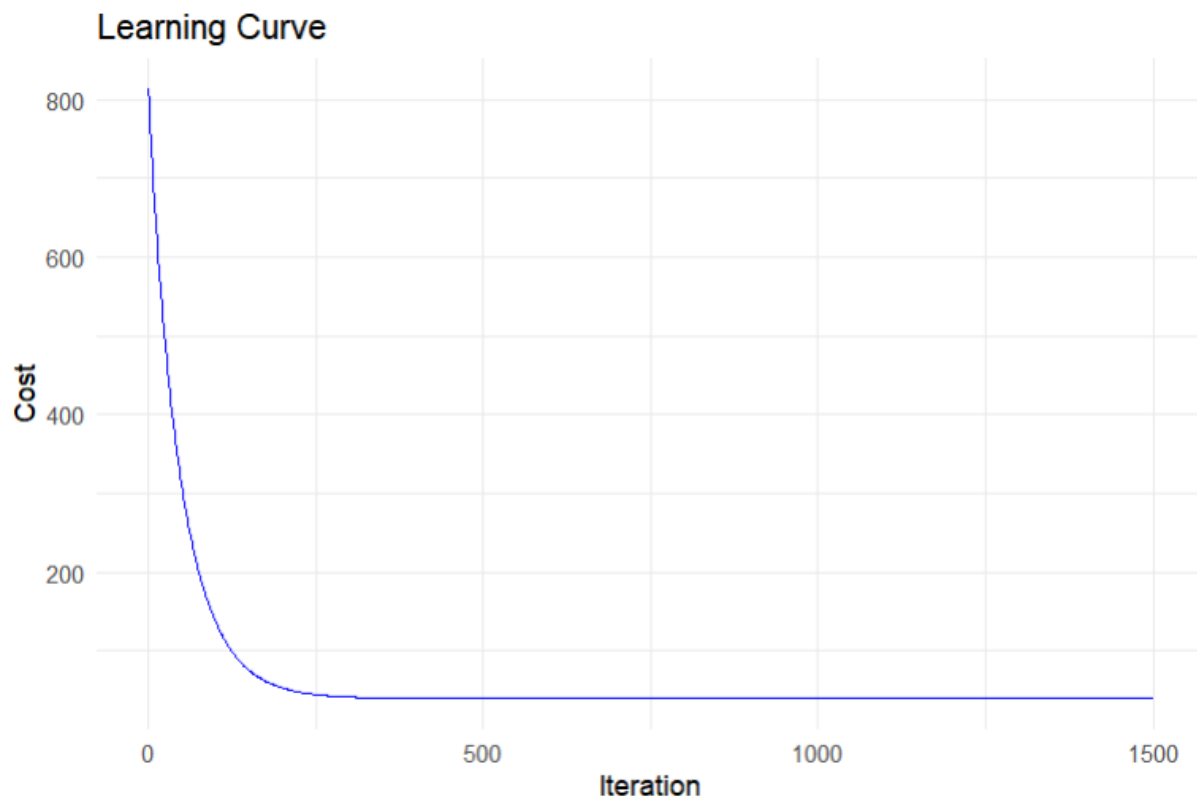
cost 함수 정의 theta_vector모델(내가 작성한 모델)이 얼마나 잘 예측하는지 측정하는 함수. 실제 부동산 데이터를 예측하는 성능을 이후 lm모델과 비교하는데 사용함

```
num_iter <- 1500  
alpha <- 0.01 # 학습률, 데이터에 따라 적절히 조정 가능  
n <- nrow(featureDF)  
costDF <- data.frame(iter = 0, t(theta_vector), cost = costFun(theta_vector))  
for (i in 1:num_iter) {  
  # 기울기 계산  
  errors <- as.matrix(featureDF) %*% theta_vector - labelVector  
  theta_update <- alpha * (t(as.matrix(featureDF)) %*% errors) / n  
  
  # theta 업데이트  
  theta_vector <- theta_vector - theta_update  
  
  # Cost 값 기록  
  costDF <- rbind(costDF, c(i, theta_vector, costFun(theta_
```

```
vector)))  
}
```

경사하강법을 통해 예측수행

```
# Cost 값 시각화  
ggplot(costDF, aes(x = iter, y = cost)) +  
  geom_line(color = "blue") +  
  labs(title = "Learning Curve", x = "Iteration", y = "Cost") +  
  theme_minimal()
```



iteration이 250안쪽에서 cost가 수렴했다.

```
# 예측 값 계산  
predicted_values <- as.matrix(featureDF) %*% theta_vector
```

```

# MSE 계산
mse <- mean((predicted_values - labelVector)^2)

# MAE 계산
mae <- mean(abs(predicted_values - labelVector))

# R^2 계산
ss_total <- sum((labelVector - mean(labelVector))^2)
ss_residual <- sum((labelVector - predicted_values)^2)
r_squared <- 1 - (ss_residual / ss_total)

# 성능 지표 출력
cat("Mean Squared Error (MSE):", mse, "\n")
cat("Mean Absolute Error (MAE):", mae, "\n")
cat("R-squared:", r_squared, "\n")

```

```

> # 성능 지표 출력
> cat("Mean Squared Error (MSE):", mse, "\n")
Mean Squared Error (MSE): 79.21798
> cat("Mean Absolute Error (MAE):", mae, "\n")
Mean Absolute Error (MAE): 6.176524
> cat("R-squared:", r_squared, "\n")
R-squared: 0.5710744

```

Task 5: lm함수를 사용해서 학습한 linear regression 모델과 Task 4 에서 얻은 linear regression 모델을 비교해 보자 학습한 모델을 비교해 보고 모델의 성능도 비교해보자

```

lm_model <- lm(price ~ house_age_norm + distance_to_mrt_norm + num_convenience_norm + latitude_norm + longitude_norm, data = estate)
# 예측 값 계산
predicted_values_lm <- predict(lm_model, newdata = estate)

```



```
# 실제 값
actual_values <- estate$price
```

동일한 데이터에, 정규화를 마친 변수만을 사용하여 예측을 수행한다.

```
# MSE 계산
mse_lm <- mean((predicted_values_lm - actual_values)^2)

# MAE 계산
mae_lm <- mean(abs(predicted_values_lm - actual_values))

# R^2 계산
ss_total_lm <- sum((actual_values - mean(actual_values))^2)
ss_residual_lm <- sum((actual_values - predicted_values_lm)^2)
r_squared_lm <- 1 - (ss_residual_lm / ss_total_lm)

# 성능 지표 출력
cat("Mean Squared Error (MSE):", mse_lm, "\n")
cat("Mean Absolute Error (MAE):", mae_lm, "\n")
cat("R-squared:", r_squared_lm, "\n")
```

```
> # 성능 지표 출력
> cat("Mean Squared Error (MSE):", mse_lm, "\n")
Mean Squared Error (MSE): 79.20185
> cat("Mean Absolute Error (MAE):", mae_lm, "\n")
Mean Absolute Error (MAE): 6.174869
> cat("R-squared:", r_squared_lm, "\n")
R-squared: 0.5711617
```

이전 직접 cost함수로 계산했던 예측과 거의 비슷한 결과가 나온다.